

GUI Implementation Technical Documentation

Table of Contents

1. [Theme System](#)
2. [Custom Components](#)
3. [Layout Management](#)
4. [Responsive Design](#)

Theme System

1. Color Management (ThemeColors)

```
public class ThemeColors {  
    // Primary Colors  
    public static final Color PRIMARY = new Color(59, 130, 246); //  
    Vibrant blue  
    public static final Color PRIMARY_DARK = new Color(37, 99, 235); //  
    Deeper blue  
    public static final Color PRIMARY_LIGHT = new Color(219, 234, 254);  
    // Light blue  
    public static final Color PRIMARY_HOVER = new Color(29, 78, 216); //  
    Hover state  
    public static final Color PRIMARY_ACTIVE = new Color(30, 64, 175);  
    // Active state  
  
    // Status Colors  
    public static final Color SUCCESS = new Color(16, 185, 129); //  
    Emerald green  
    public static final Color WARNING = new Color(245, 158, 11); //  
    Amber  
    public static final Color DANGER = new Color(239, 68, 68); // Red  
    public static final Color INFO = new Color(14, 165, 233); // Sky  
    blue  
  
    // Background & Surface Colors  
    public static final Color BACKGROUND = new Color(249, 250, 251); //  
    Off-white  
    public static final Color SURFACE = new Color(255, 255, 255); //  
    Pure white  
    public static final Color SURFACE_VARIANT = new Color(247, 247,  
    248); // Off-white variant  
  
    // Text Colors  
    public static final Color TEXT_PRIMARY = new Color(17, 24, 39); //  
    Near-black  
    public static final Color TEXT_SECONDARY = new Color(107, 114, 128);  
    // Gray
```

```
        public static final Color TEXT_DISABLED = new Color(156, 163, 175);  
        // Light gray  
    }
```

2. Font System (ThemeFonts)

```
public class ThemeFonts {  
    private static final String PRIMARY_FONT = "Inter";  
    private static final String SECONDARY_FONT = "Roboto";  
  
    // Regular fonts  
    public static final Font REGULAR_SMALL = new Font(PRIMARY_FONT,  
Font.PLAIN, 12);  
    public static final Font REGULAR_MEDIUM = new Font(PRIMARY_FONT,  
Font.PLAIN, 14);  
    public static final Font REGULAR_LARGE = new Font(PRIMARY_FONT,  
Font.PLAIN, 16);  
  
    // Bold fonts  
    public static final Font BOLD_MEDIUM = new Font(PRIMARY_FONT,  
Font.BOLD, 14);  
    public static final Font BOLD_SECTION = new Font(PRIMARY_FONT,  
Font.BOLD, 18);  
    public static final Font BOLD_TITLE = new Font(PRIMARY_FONT,  
Font.BOLD, 26);  
}
```

3. Size Constants (ThemeSizes)

```
public class ThemeSizes {  
    // Component sizes  
    public static final int BUTTON_HEIGHT = 40;  
    public static final int TEXT_FIELD_HEIGHT = 40;  
    public static final int TABLE_ROW_HEIGHT = 40;  
  
    // Padding and margins  
    public static final int PADDING_SMALL = 8;  
    public static final int PADDING_MEDIUM = 16;  
    public static final int PADDING_LARGE = 24;  
  
    // Border radius  
    public static final int BORDER_RADIUS_SMALL = 4;  
    public static final int BORDER_RADIUS_MEDIUM = 8;  
    public static final int BORDER_RADIUS_LARGE = 12;  
}
```

Custom Components

1. Base Components

StyledButton

A modern button component with hover effects and consistent styling:

```
public class StyledButton extends JButton {
    private boolean isActive;

    public StyledButton(String text) {
        super(text);
        initialize();
    }

    private void initialize() {
        setFont(ThemeFonts.BOLD_SECTION);
        setForeground(Color.WHITE);
        setBackground(ThemeColors.PRIMARY);
        setFocusPainted(false);
        setBorderPainted(false);
        setCursor(new Cursor(Cursor.HAND_CURSOR));

        addMouseListener(new ButtonMouseListener());
        setBorder(new RoundedBorder(15));
    }

    // Supports active state
    public void setActive(boolean active) {
        isActive = active;
        setBackground(active ? ThemeColors.PRIMARY_DARK :
ThemeColors.PRIMARY);
    }
}
```

StyledTable

A customized table with modern header and cell styling:

```
public class StyledTable<T> extends JTable {
    private List<T> data;

    public StyledTable(String[] columns) {
        super(new DefaultTableModel(columns, 0));
        setupTableAppearance();
    }

    private void setupTableAppearance() {
        setFont(ThemeFonts.REGULAR_MEDIUM);
        setRowHeight(ThemeSizes.TABLE_ROW_HEIGHT);
    }
}
```

```

        setShowGrid(true);
        setGridColor(ThemeColors.BORDER);

        // Header styling
        JTableHeader header = getTableHeader();
        header.setFont(ThemeFonts.BOLD_MEDIUM);
        header.setBackground(ThemeColors.PRIMARY);
        header.setForeground(ThemeColors.TEXT_PRIMARY);
    }
}

```

StyledTextField

A modern text field with placeholder support:

```

public class StyledTextField extends JTextField {
    private String placeholder;
    private boolean isPlaceholder;

    public StyledTextField(String placeholder) {
        this.placeholder = placeholder;
        setFont(ThemeFonts.REGULAR_MEDIUM);
        setForeground(ThemeColors.TEXT_PRIMARY);
        setBackground(ThemeColors.SURFACE);
        setCaretColor(ThemeColors.PRIMARY);
    }
}

```

2. Dialog Components

ModernDialog

A utility class for creating consistently styled dialogs:

```

public class ModernDialog {
    public static void showMessage(Component parent, String message,
String title, int messageType) {
        JOptionPane pane = ThemeManager.createStyledOptionPane(
            message, title, messageType, JOptionPane.DEFAULT_OPTION,
            null);

        JDialog dialog = pane.createDialog(parent, title);
        styleDialog(dialog);
        dialog.setVisible(true);
    }

    private static void styleDialog(JDialog dialog) {
        dialog.setBackground(ThemeColors.SURFACE);
    }
}

```

```

        styleAllButtonsInContainer(dialog);
    }
}

```

MessageDialog

Specialized dialog for showing different types of messages:

```

public class MessageDialog {
    public static void showError(Component parent, String message) {
        showDialog(parent, "Error", message, JOptionPane.ERROR_MESSAGE);
    }

    public static void showSuccess(Component parent, String message) {
        showDialog(parent, "Success", message,
JOptionPane.INFORMATION_MESSAGE);
    }
}

```

3. Layout Components

BasePanel

Base class for all panels with common functionality:

```

public abstract class BasePanel extends JPanel {
    protected MainFrame mainFrame;
    protected PharmacyService service;

    public BasePanel(MainFrame mainFrame) {
        setBackground(ThemeColors.BACKGROUND);
        setBorder(createBorder());
        this.mainFrame = mainFrame;
        this.service = mainFrame.getService();
    }

    protected abstract void initializeComponents();

    public void refresh() {
        removeAll();
        initializeComponents();
        revalidate();
        repaint();
    }
}

```

ActionButton

A specialized button for dashboard actions:

```
public class ActionButton extends JPanel {
    private final JLabel iconLabel;
    private final JLabel titleLabel;
    private final JLabel descriptionLabel;

    public ActionButton(String title, String description, Icon icon) {
        setLayout(new BorderLayout(10, 0));
        setBackground(ThemeColors.SURFACE);
        setCursor(new Cursor(Cursor.HAND_CURSOR));

        // Components initialization
        iconLabel = new JLabel(icon);
        titleLabel = new JLabel(title);
        descriptionLabel = new JLabel(description);

        // Styling
        titleLabel.setFont(ThemeFonts.BOLD_MEDIUM);
        descriptionLabel.setFont(ThemeFonts.REGULAR_SMALL);
    }
}
```

Theme Management

ThemeManager

Centralized theme management and UI styling:

```
public class ThemeManager {
    private static ThemeManager instance;
    private String currentTheme;
    private Map<String, Theme> themes;

    public static void initializeModernUI() {
        // Apply standard styling
        UIManager.put("Button.background", ThemeColors.PRIMARY);
        UIManager.put("Button.foreground", Color.WHITE);
        UIManager.put("Panel.background", ThemeColors.SURFACE);

        // Initialize all components
        modernizeOptionPanels();
        modernizeTables();
        modernizeFileChoosers();
    }

    public static void applyTextComponentStyling(JTextComponent
```

```

component) {
    component.setFont(ThemeFonts.REGULAR_MEDIUM);
    component.setForeground(ThemeColors.TEXT_PRIMARY);
    component.setBackground(ThemeColors.SURFACE);
}
}

```

Usage Examples

Creating a Dashboard Panel

```

public class DashboardPanel extends BasePanel {
    public DashboardPanel(MainFrame mainFrame) {
        super(mainFrame);
        initializeComponents();
    }

    @Override
    protected void initializeComponents() {
        setLayout(new BorderLayout(15, 15));
        setBackground(ThemeColors.BACKGROUND);

        // Add components with consistent styling
        add(createHeaderPanel(), BorderLayout.NORTH);
        add(createContentPanel(), BorderLayout.CENTER);
    }

    private JPanel createHeaderPanel() {
        JPanel header = new JPanel(new BorderLayout());
        header.setBackground(ThemeColors.SURFACE);
        header.setBorder(new CompoundBorder(
            new LineBorder(ThemeColors.BORDER, 1),
            new EmptyBorder(15, 15, 15, 15)
        ));
        return header;
    }
}

```

This documentation now accurately reflects the actual implementation in your codebase, including all custom components, theme management, and styling details.